SERVICE-LAYER QUALITY — REFERENCE

API & Serverless Test Strategy

Northwind Financial · Global Quality Engineering

Layer: Service / API tier
Applies to: REST · GraphQL · event-driven

STRATEGY

API Testing — verify the interface contract

Under test: an always-on service endpoint (REST / GraphQL / gRPC).

- Functional / happy-path — endpoints return correct data
- Schema & contract — OpenAPI / Pact, no breaking changes
- Negative & boundary — bad input → correct 4xx
- Auth & authz — token, scope, **tenant isolation**
- Idempotency, pagination, filtering
- Performance — latency, throughput, rate limits
- Security — OWASP API Top 10 (BOLA, mass assignment)

Robot Framework · RequestsLibrary · Postman / Newman · REST Assured · Pact · Schemathesis · k6 · OWASP ZAP

Serverless Testing — everything above, plus the runtime

Under test: event-driven functions + managed services (Lambda / Azure Functions).

- Trigger / event-shape — HTTP, queue, storage, timer, stream
- Handler unit — mock event object + mocked cloud SDK
- Local integration — emulate the cloud services
- **Idempotency & retries** — DLQ + partial-batch failures
- Cold-start & concurrency — perf under scale-from-zero
- IAM least-privilege — function permissions are correctness
- Eventual consistency — async assertions on side effects

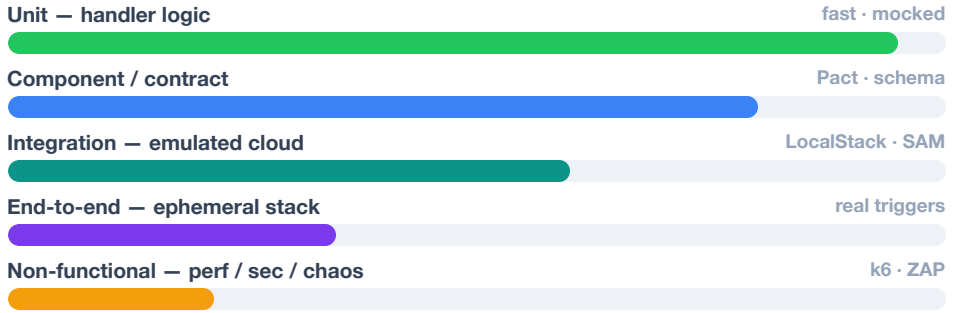
LocalStack · AWS SAM CLI · Serverless Framework · moto · Azurite · EventBridge schema registry · AWS Powertools

COVERAGE MATRIX — SAME CONCERN, WHAT EACH LAYER ADDS

blue = API testing · teal = serverless adds

QUALITY CONCERN	API TESTING CHECKS	SERVERLESS TESTING ADDS
Correctness	status · schema · data	+ event shape per trigger
Reliability	retries · timeouts	+ idempotency · DLQ · partial batch
Auth & access	token · scope · tenant	+ IAM least-privilege
Performance	latency · throughput · limits	+ cold start · concurrency caps
State	DB side-effects	+ eventual consistency · ephemeral
Security	OWASP API Top 10	+ function perms · secrets · cost guardrails
Where it runs	shared live test env	+ local emulation → ephemeral cloud stack

WHERE THE TESTS RUN — THE PYRAMID



Heavy fast base (handler units), thin slow tip (E2E on a real stack). Serverless pushes **integration** left via local cloud emulation.

Serverless = API testing + runtime

A serverless API needs **both** layers: the contract tests prove the interface is right; the runtime tests prove events, retries, cold starts and IAM behave. They catch different defect classes — neither replaces the other.

In this lab: `api-contract` (Pact/schema), `performance` (k6) and `security` (ZAP) pipelines already cover the API tier — serverless adds an emulated-integration + idempotency suite to the same CI.